

SKILL - 6

Name: Simran Jangid

Rollno: 2520030166

Section: 8

Aim

To create process escape-sequence handling that preserves escaped spaces and special symbols during command parsing, validates parser output, and supports complex inputs. Also, to create child processes that execute programs, pass arguments, handle execution errors, and manage the parent process correctly.

Objective

- To create process escape sequences.
- To handle escaped spaces in command input.
- To escape and preserve special symbols.
- To validate parser output for complex inputs.
- To test command parsing with different inputs.
- To create child processes using process creation.
- To execute programs from a child process.
- To pass arguments to the executed program.
- To handle execution errors.
- To manage and wait for the parent process.

Problem Statement

Design and implement a command-processing program that can recognize escaped spaces and special symbols without incorrectly splitting them. The program should validate the parsed arguments and then create a child process to execute a program with the parsed arguments. The parent process should manage the child process and report whether execution was successful.

Theory

Command processing requires an input string to be converted into meaningful arguments before a program can be executed. Spaces normally separate arguments, but an escaped space should be preserved as part of the same argument. Similarly, escaped special symbols should remain as literal characters instead of being interpreted as separators or control symbols.

1. Process Escape Sequences

An escape sequence provides a way to indicate that a character should be treated literally. For example, a backslash before a space can represent an escaped space. The parser recognizes the escape character and preserves the following character in the current argument.

2. Escaped Spaces

A normal space separates two arguments. An escaped space does not create a new argument. For example, the input 'hello world' can represent two arguments, while 'hello\ world' represents one argument containing the space.

3. Escaping Special Symbols

Special symbols may have a meaning in command processing. An escape character can be used to preserve such symbols as literal data. The parser therefore removes the escape marker while keeping the character that follows it.

4. Character Preservation

The parser must preserve the order and value of characters that form each argument. When an escaped character is encountered, the escape marker is not copied to the argument; the escaped character itself is stored.

5. Parser Output Validation

After parsing, the resulting arguments should be checked to make sure that escaped spaces and special symbols were preserved correctly. The number of arguments and their contents can be displayed for validation.

6. Child Processes

A child process is a new process created by a parent process. In C on Unix-like systems, the `fork()` system call creates a child process. After creation, both processes continue from the point following `fork()`, but the return value allows the program to distinguish between the parent and child.

6.1 Process Creation

The `fork()` function creates a separate child process. A negative return value indicates that process creation failed, zero identifies the child process, and a positive value in the parent represents the child process ID.

6.2 Program Execution

The `exec` family of functions replaces the child process image with the requested program. The child can therefore execute another program using the arguments produced by the parser.

6.3 Passing Arguments

The parsed command is stored as an argument array. This array can be passed to an execution function so that the launched program receives the same arguments produced by the parser.

6.4 Execution Errors

If an `exec` function returns, it means the requested program could not be executed successfully. The child process should report the error and terminate without continuing as if execution had succeeded.

6.5 Parent Process

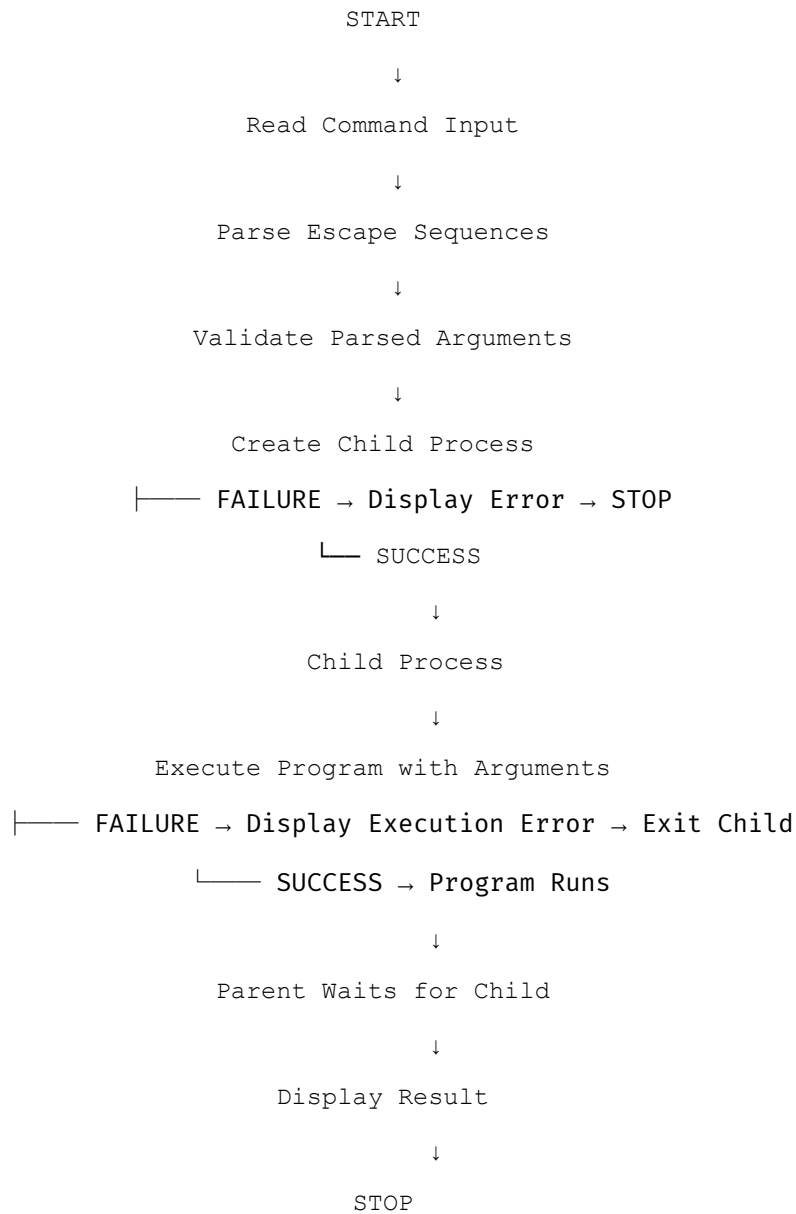
The parent process can use `waitpid()` to wait for the child to finish. This allows the parent to manage the child process and report the child's termination status.

7. Algorithm

1. Start the program.
2. Read a command string containing normal and escaped characters.
3. Initialize an empty argument list and a current argument buffer.
4. Read each character from the command.
5. If a backslash is found, preserve the next character as a literal character.
6. If an unescaped space is found, terminate the current argument.
7. Otherwise, add the character to the current argument.
8. Store the final argument in the argument list.
9. Validate the number and contents of the parsed arguments.
10. Create a child process using `fork()`.
11. If `fork()` fails, display the process creation error and terminate.
12. In the child process, execute the required program using `execvp()`.
13. If execution fails, display the execution error and terminate the child.
14. In the parent process, wait for the child using `waitpid()`.
15. Check the child's termination status.
16. Display the result of command launching.
17. Stop the program.

Control Flow

The complete control flow is represented below:



8. Data Used

A character array is used to store the input command. A two-dimensional character array stores the parsed arguments. Integer variables are used for indexes, argument counts, and process status values.

9. Operations Used

- Escape-character detection.
- Character-by-character parsing.
- Argument separation using unescaped spaces.
- Preservation of escaped characters.
- Process creation using fork().
- Program execution using execvp().
- Argument passing through the argument array.
- Parent-child synchronization using waitpid().
- Error handling using perror().

10. Test Inputs

- hello world
- hello\ world
- file\ name.txt
- hello\@test
- echo Hello\ World

11. Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 20
#define MAX_LEN 100

int parseCommand(char *input, char *args[]) {
    static char buffer[500];
    int argc = 0, j = 0;
    int escaped = 0;

    strcpy(buffer, input);

    for (int i = 0; buffer[i] != '\0'; i++) {
        char ch = buffer[i];

        if (escaped) {
            buffer[j++] = ch;
            escaped = 0;
        } else if (ch == '\\') {
            escaped = 1;
        } else if (ch == ' ') {
            if (j > 0) {
                buffer[j++] = '\0';
            }
        }
    }
    buffer[j] = '\0';
    return j;
}
```

```

        args[argc++] = &buffer[j - 1];
    }
} else {
    buffer[j++] = ch;
}
}

if (j > 0) {
    buffer[j] = '\0';
    args[argc++] = &buffer[j - 1];
}

args[argc] = NULL;
return argc;
}

int main() {
    char input[] = "echo Hello\\ World";
    char *args[MAX_ARGS];

    printf("Skill - 6: Parser and Process Execution\n");
    printf("-----\n");

    printf("Input: %s\n", input);

    int argc = parseCommand(input, args);

    printf("Parsed arguments:\n");
    for (int i = 0; i < argc; i++) {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child process created.\n");
        printf("Executing command...\n");
        execvp(args[0], args);
        perror("execvp");
        exit(1);
    } else {
        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status) && WEXITSTATUS(status) == 0)
            printf("Parent: Child executed successfully.\n");
        else
            printf("Parent: Child execution failed.\n");
    }

    return 0;
}

```

12. Program Explanation

12.1 Parser Function

The `parseCommand()` function reads the input character by character. A backslash sets an escape flag, causing the next character to be copied literally. An unescaped space ends the current argument.

12.2 Argument Array

The `args` array stores pointers to the parsed arguments. A `NULL` value is placed at the end so that the array can be directly passed to `execvp()`.

12.3 `fork()`

`fork()` creates a child process. The parent receives the child's process ID, while the child receives zero. A negative value indicates that creation failed.

12.4 `execvp()`

The child process uses `execvp()` to replace itself with the requested program. The command name and parsed arguments are passed through the `args` array. If `execvp()` fails, `perror()` displays the reason for the failure.

12.5 `waitpid()`

The parent process calls `waitpid()` to wait until the child process finishes. The returned status is checked to determine whether the child terminated successfully.

13. Sample Output

The following is an example of the expected program output:

```
Skill - 6: Parser and Process Execution
-----
Input: echo Hello\ World
Parsed arguments:
Argument 1: echo
Argument 2: Hello World
Child process created.
Executing command...
Hello World
Parent: Child executed successfully.
```

14. Output Screenshot

Paste the screenshot of the program output in the space provided below.

```
Skill - 6: Parser and Process Execution
-----
Input: echo Hello\ World
Parsed arguments:
Argument 1: echo
Argument 2: Hello World
Child process created.
Executing command...
Hello World
Parent: Child executed successfully.
7rainreddy7@Gunapatis-MacBook-Air ~ %
```

15. Testing

- Tested an input containing a normal space.
- Tested an escaped space and verified that it remained inside one argument.
- Tested an escaped special symbol.
- Verified that parsed arguments were displayed correctly.
- Verified that a child process was created.
- Verified that the child executed the requested program.
- Verified that arguments were passed to the child process.
- Verified that the parent waited for the child process.

16. Test Cases

Test Case	Input	Expected Result
1	hello world	Two separate arguments are produced.
2	hello\ world	One argument containing a space is produced.
3	file\ name.txt	File name is preserved as one argument.
4	hello\@test	The special symbol is preserved.
5	echo Hello\ World	Child executes echo with Hello World as one argument.
6	Invalid program	Execution error is reported.

17. Observation

- Escaped spaces were preserved as part of the same argument.
- Escaped special characters were retained during parsing.
- The parser produced a validated argument list.
- A child process was successfully created.
- The child process executed the requested program.
- Arguments were passed correctly to the executed program.
- The parent process waited for the child to finish.
- Execution errors could be reported when program launching failed.

18. Result

Process escape-sequence handling and command parsing were successfully implemented. Escaped spaces and special symbols were preserved, parsed arguments were validated, and a child process was created to execute a program with the parsed arguments. The parent process successfully managed the child and checked its execution status.

19. Conclusion

The practical demonstrated the connection between command parsing and process management. Escape-sequence processing allows complex command input to be represented correctly, while child-process creation and program execution allow the parsed command to be launched. Together, these operations form an important part of command interpreters and shell-based systems.